Companion to Bird & de Moor, Algebra of Programming

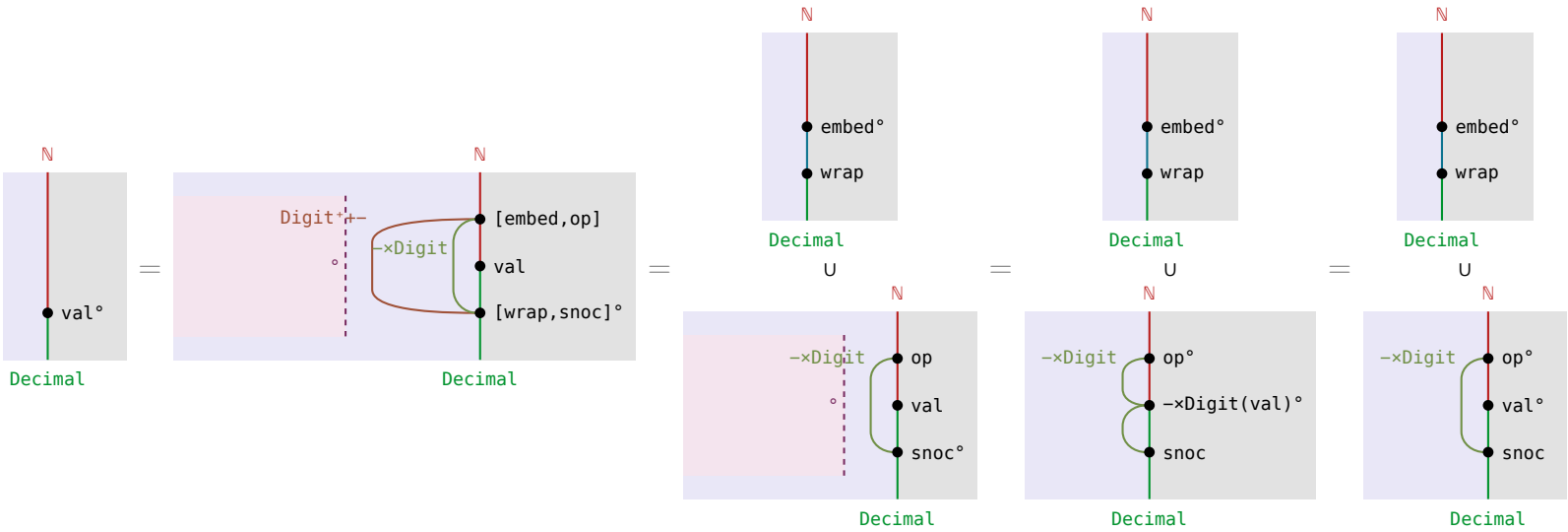
§6. Recursive Programs

§6.1. Digits of a number

`val° = (embed° wrap) ∪ (op° −×Digit(val°) snoc)` (6.1a)

a number is read back into digits either as one nonzero digit, or by splitting off its last digit and reading back the rest

the converse of `val`, which reads digits as a number



(a) (b) (c) (d) (e)

(a)	`val°`	`val` undoes `wrap` or `snoc`, reads the front with `val` and combines by `embed` or `op`, so its converse runs that backwards
(b)	= `([wrap,snoc]° Digit⁺+−(−×Digit(val)) [embed,op])°`	undoing `wrap` or `snoc` first gives two cases: a decimal and its last digit read by `val` then `op`, or one digit read by `embed`
(c)	= `(wrap° embed)° ∪ (snoc° −×Digit(val) op)°`	the converse of each case is its steps undone in reverse: undo `op`, undo reading the front, then `snoc`; or undo `embed`, then `wrap`
(d)	= `(embed° wrap) ∪ (op° −×Digit(val)° snoc)`	undoing the read of the front while keeping the last digit is `val°` on the front alone
(e)	= `(embed° wrap) ∪ (op° −×Digit(val°) snoc)`	

(6.1b)

`op°(m,p) ⟺ π₁(p) = m/10 ∧ π₂(p) = m % 10`

`op°` splits a number into its quotient and remainder by 10

`(∃p. 0 < π₁ p ∧ op°(m,p)) ⟺ 10 ≤ m`

`op°` gives a pair with a nonzero first component exactly at the numbers with two or more digits

`(∃d. embed°(m,d)) ⟺ 0 < m ∧ m < 10`

`embed°` gives a digit exactly at the one-digit numbers

§6.2. Least fixed points

(6.2a)

φ a **monotonic** mapping of the hom-set A→B into itself: X⊑Y⟹φ(X)⊑φ(Y) .

(μX : φ(X)) the least X : A→B with φ(X)⊑X .

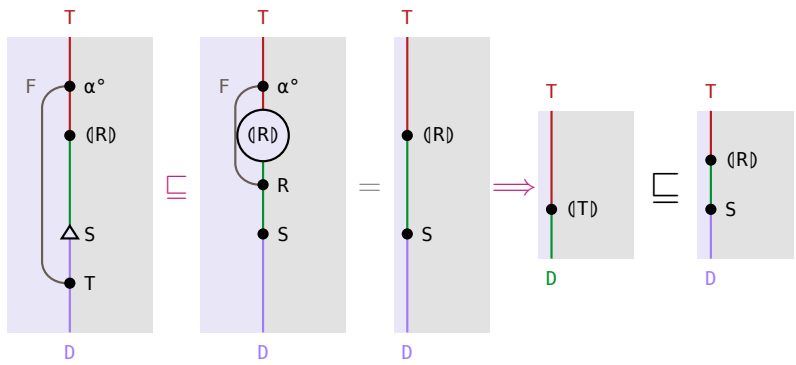
(6.2b)

the law	what it says
φ(Y) ⊑ Y ⟹ (μX : φ(X)) ⊑ Y	to bound (μX : φ(X)) above, exhibit one Y the body does not grow past — the half §Section 6.3 and every chapter after it uses
Monotonic(φ) ⟹ φ(μX : φ(X)) = (μX : φ(X))	**Knaster–Tarski**: the least solution of φ(X)⊑X already solves φ(X)=X, so the least prefix point and the least fixed point are one relation
α° F(X) R ⊑ X ⟹ ⦇R⦈ ⊑ X (6.2)	a relation X that contains one unfolding of the fold's recursion at X contains the fold
X ⊑ α° F(X) R ⟹ X ⊑ ⦇R⦈ (6.3)	a relation X contained in one unfolding of the fold's recursion at X is contained in the fold

F(S) T ⊑ R S ⟹ ⦇T⦈ ⊑ ⦇R⦈ S (6.2c)

if **S** followed by **R** absorbs **T** after **F(S)**, the fold of **T** is below the fold of **R** followed by **S**

X≜⦇R⦈S

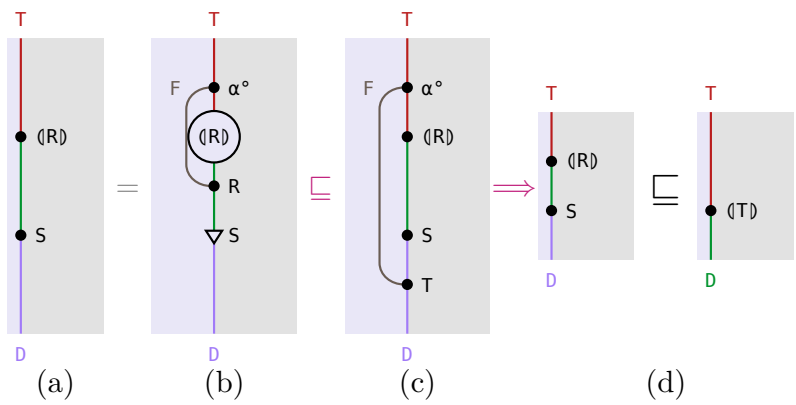


(a)	α° F(⦇R⦈) F(S) T	F(S)T⊑RS
(b)	⊑ α° F(⦇R⦈) R S	F(⦇R⦈)R=α⦇R⦈, α°α=𝟙 — ≜ reduce
(c)	= ⦇R⦈ S	⦇R⦈S is a prefix point of α°F(−)T, and ⦇T⦈ is the least one — (6.2) α°F(X)T⊑X ⟹ ⦇T⦈⊑X
(d)	⟹ α° F(⦇R⦈ S) T ⊑ ⦇R⦈ S ⟹ ⦇T⦈ ⊑ ⦇R⦈ S	

R S ⊑ F(S) T ⟹ ⦇R⦈ S ⊑ ⦇T⦈ (6.2d)

if **R** followed by **S** is absorbed by **F(S)** followed by **T**, the fold of **R** followed by **S** is below the fold of **T**

X≜⦇R⦈S



(a)	⦇R⦈ S	α⦇R⦈=F(⦇R⦈)R, α°α=𝟙 — ≜ reduce
(b)	= α° F(⦇R⦈) R S	RS⊑F(S)T
(c)	⊑ α° F(⦇R⦈ S) T	⦇R⦈S is a postfix point of α°F(−)T, and ⦇T⦈ is the greatest one — (6.3) X⊑α°F(X)T ⟹ X⊑⦇T⦈
(d)	⟹ ⦇R⦈ S ⊑ α° F(⦇R⦈ S) T ⟹ ⦇R⦈ S ⊑ ⦇T⦈	

§6.3. Hylomorphisms

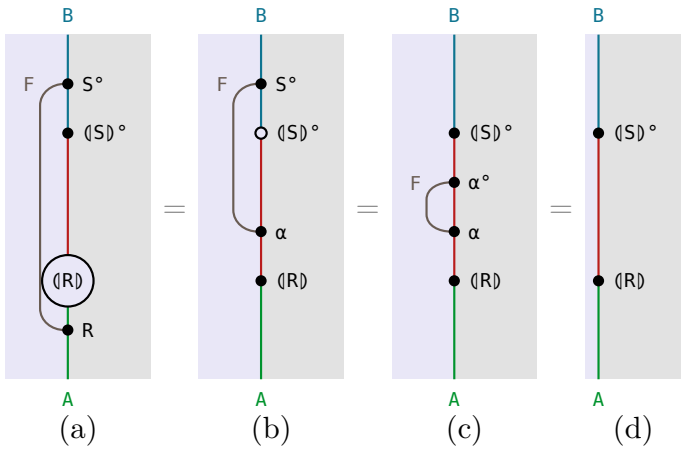
⦇S⦈° ⦇R⦈ = (μX : S° F(X) R) (6.3a)

hylomorphism theorem: a hylomorphism is the least fixed point of a certain recursion equation

S° F(⦇S⦈° ⦇R⦈) R = ⦇S⦈° ⦇R⦈

hylomorphism theorem: a prototypical 'divide and conquer' scheme — the term S° represents the decomposition stage, F(⦇S⦈°⦇R⦈) the stage of solving the subproblems recursively, and R the recombination stage; R : FA→A, S : FB→B, α : FT→T initial

the body at ⦇S⦈°⦇R⦈

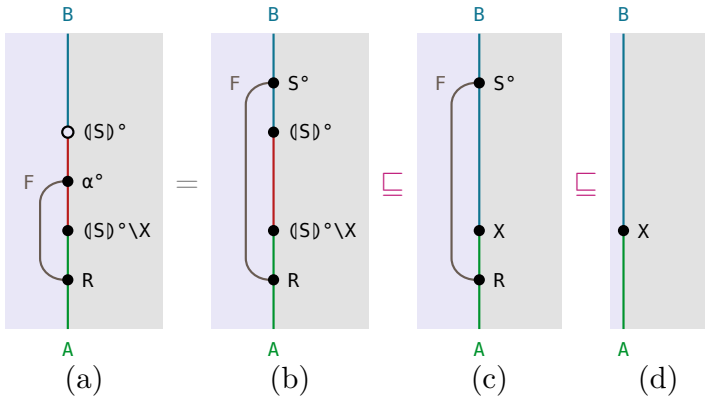


(a)	S° F(⦇S⦈°) F(⦇R⦈) R	F(⦇R⦈)R=α⦇R⦈ — ≜ reduce
(b)	= S° F(⦇S⦈°) α ⦇R⦈	⦇S⦈°α°=S°F(⦇S⦈°) — ≜ reduce, relator-laws
(c)	= ⦇S⦈° α° α ⦇R⦈	α iso
(d)	= ⦇S⦈° ⦇R⦈	

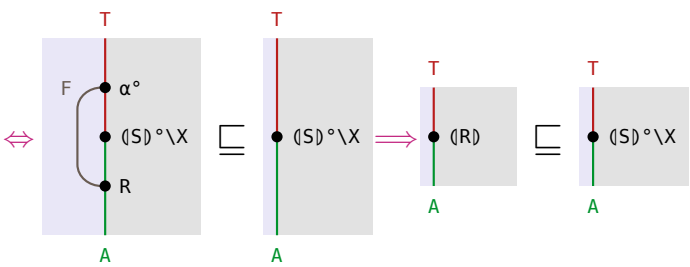
S° F(X) R ⊑ X ⟹ ⦇S⦈° ⦇R⦈ ⊑ X

hylomorphism theorem: by Knaster–Tarski, the hylomorphism ⦇S⦈°⦇R⦈ is included in X if X satisfies the associated recursion inequation

Y:=⦇S⦈°\X



(a)	⦇S⦈° α° F(⦇S⦈° \ X) R	⦇S⦈°α°=S°F(⦇S⦈°)
(b)	= S° F(⦇S⦈° (⦇S⦈° \ X)) R	⦇S⦈°(⦇S⦈°\X)⊑X — adj-all
(c)	⊑ S° F(X) R	S°F(X)R⊑X
(d)	⊑ X	S·⊣S\ — adj-all

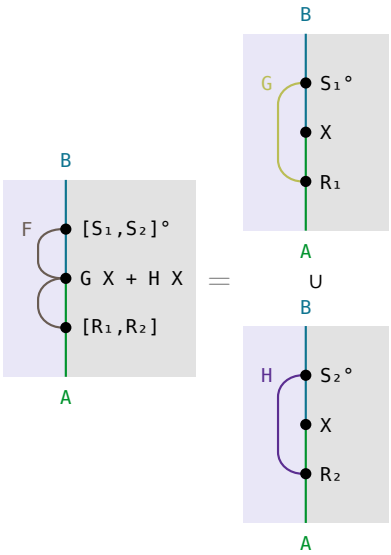


(a)	⇔ S° F(X) R ⊑ X ⟹ α° F(⦇S⦈° \ X) R ⊑ ⦇S⦈° \ X	⦇R⦈=(μX : α°F(X)R) — ≜ reduce, (6.2b), adj-all
(b)	⟹ S° F(X) R ⊑ X ⟹ ⦇R⦈ ⊑ ⦇S⦈° \ X	

$$(\forall x.\ \forall y.\ \forall X.\ F(X) = G(X) + H(X)) \Longrightarrow (\!|[S_1,S_2]|\!)^\circ\ (\!|[R_1,R_2]|\!) = (\mu X : (S_1{}^\circ\ G(X)\ R_1) \cup (S_2{}^\circ\ H(X)\ R_2)) \tag{6.3b}$$

when both algebras are case splits over the same sum, the recursion runs each case on its own and unites the results

the body of Theorem 6.2 at F(X)=G(X)+H(X)



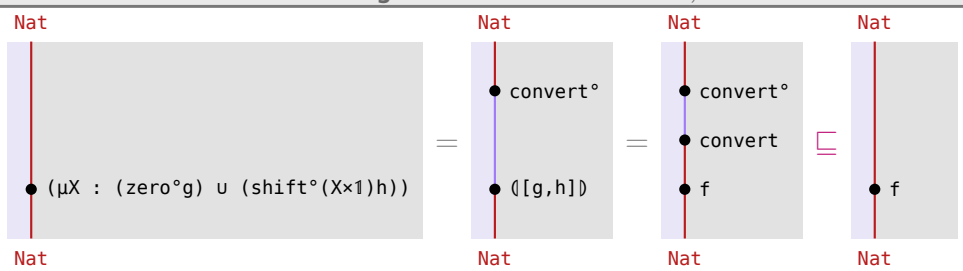
(a) (b)

(a)	$[S_1,S_2]^\circ\ (G(X) + H(X))\ [R_1,R_2]$	$[S,T]=(S+T)\nabla$ on both sides gives $\nabla^\circ(S_1{}^\circ G(X)R_1+S_2{}^\circ H(X)R_2)\nabla$; the + stands for the codiagonals ∇°, ∇, which are not drawn $\nabla^\circ(P+Q)\nabla=P\cup Q$ — coproduct
(b)	$= (S_1{}^\circ\ G(X)\ R_1) \cup (S_2{}^\circ\ H(X)\ R_2)$	

§6.4. Fast exponentiation and modulus computation

$$\text{zero } f = g \wedge \text{shift } f = (f \times \mathbb{1})\ h \Longrightarrow (\mu X : (\text{zero}^\circ\ g) \cup (\text{shift}^\circ\ (X \times \mathbb{1})\ h)) \sqsubseteq f \tag{6.4a}$$

once **f** sends **zero** to **g** and turns **shift** into **h**, the recursion that halves the argument at each step computes **f**



(a) (b) (c) (d)

(a)	$(\mu X : (\text{zero}^\circ\ g) \cup (\text{shift}^\circ\ (X \times \mathbb{1})\ h))$	$\text{convert}^\circ\ (\!|[g,h]|\!) = (\mu X : (\text{zero}^\circ\ g) \cup (\text{shift}^\circ\ (X \times \mathbb{1})\ h))$
(b)	$= \text{convert}^\circ\ (\!|[g,h]|\!)$	$\text{zero } f = g \wedge \text{shift } f = (f \times \mathbb{1})\ h \Longrightarrow \text{convert } f = (\!|[g,h]|\!)$
(c)	$= \text{convert}^\circ\ \text{convert}\ f$	Simple(convert) $\mathbb{1}\ f = f$
(d)	$\sqsubseteq f$	

$$(\mu X : (\text{zero}^\circ\ \text{one}) \cup (\text{shift}^\circ\ (X \times \mathbb{1})\ \text{op}(a))) \sqsubseteq \text{exp}(a) \tag{6.4b}$$

the recursion that halves the exponent at each step computes **a** to the power **b**

zero exp(a) = one

a to the power 0 is 1

shift exp(a) = (exp(a) × 𝟙) op(a)

a to the power 2n+d is op(a) of a to the power n and d

$$(\mu X : (\text{zero}^\circ\ \text{zero}) \cup (\text{shift}^\circ\ (X \times \mathbb{1})\ \text{op}(b))) \sqsubseteq \text{mod}(b) \tag{6.4c}$$

the recursion that halves **a** at each step computes the remainder of **a** by **b**

zero mod(b) = zero

0 mod b is 0

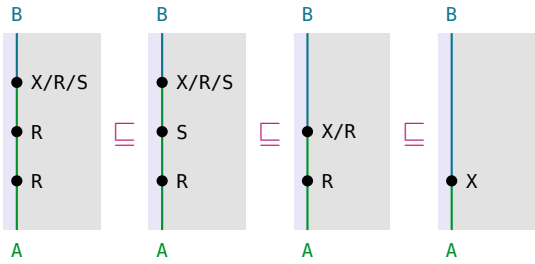
shift mod(b) = (mod(b) × 𝟙) op(b)

(2a+d) mod b is op(b) of a mod b and d

§6.5. Unique fixed points

inductive(S) ∧ R R ⊑ S R ⟹ inductive(R) (6.5a)

R is inductive when RR⊑SR for an inductive S



(a) (b) (c) (d)

(a)	(X / R / S) R R	RR⊑SR
(b)	⊑ (X / R / S) S R	division by S
(c)	⊑ (X / R) R	division by R
(d)	⊑ X	

member = 0 (6.5b)

a constant relator records no elements, so its membership is empty

member = [member,member]

a member of F+G is a member of whichever summand the value is in

member = (π₁ member) ∪ (π₂ member)

a member of F×G is a member of either component

inductive(S member) ∧ X = S F(X) R ∧ Y = S F(Y) R ⟹ X = Y (6.5c)

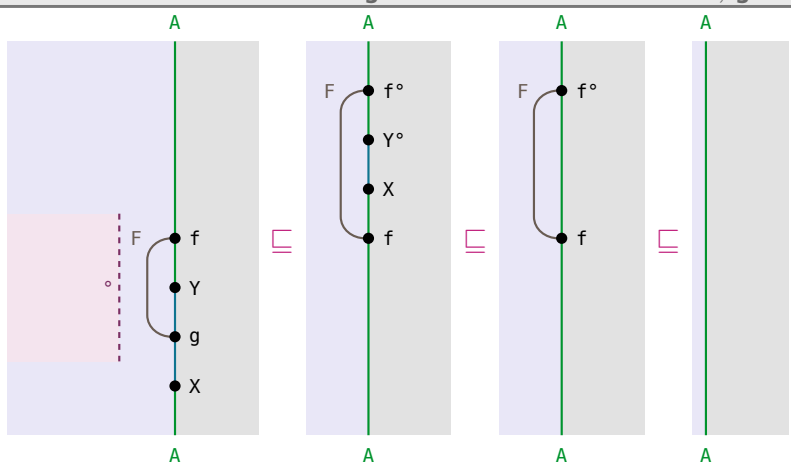
when S member is inductive, two solutions X, Y of X=SF(X)R are equal

inductive(S member) ∧ Entire(S) ∧ Entire(R) ∧ S F(X) R ⊑ X ⟹ Entire(X) (6.5d)

when S member is inductive and S, R are entire, a pre-fixed point of X↦SF(X)R is entire

inductive(g member) ∧ Map(g) ∧ Map(f) ∧ X = g F(X) f ⟹ Map(X) (6.5e)

when g member is inductive and f, g are maps, the solution of X=gF(X)f is a map

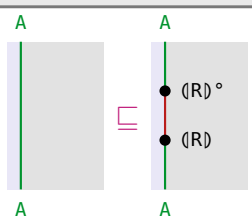


(a) (b) (c) (d)

(a)	(g F(Y) f)° X	X=gF(X)f, g simple
(b)	⊑ f° F(Y° X) f	Y°X⊑𝟙
(c)	⊑ f° f	f simple
(d)	⊑ 𝟙	

inductive(R° member) ∧ 𝟙 ⊑ R° R ⟹ 𝟙 ⊑ ⦇R⦈° ⦇R⦈ (6.5f)

when R°member is inductive and R is surjective, the fold ⦇R⦈ is surjective

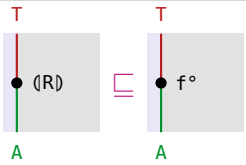


(a) (b)

(a)	𝟙	(6.5d) at S=R°
(b)	⊑ ⦇R⦈° ⦇R⦈	

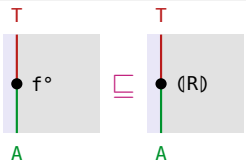
Map(f) ∧ R f ⊑ F(f) α ∧ 𝟙 ⊑ R° R ⟹ f° = ⦇R⦈ (6.5g)

a map **f** with **Rf⊑F(f)α**, for a surjective **R**, has the fold ⦇R⦈ as its converse



(a) (b)

(a)	⦇R⦈	shunting f, fusion, Rf⊑F(f)α
(b)	⊑ f°	

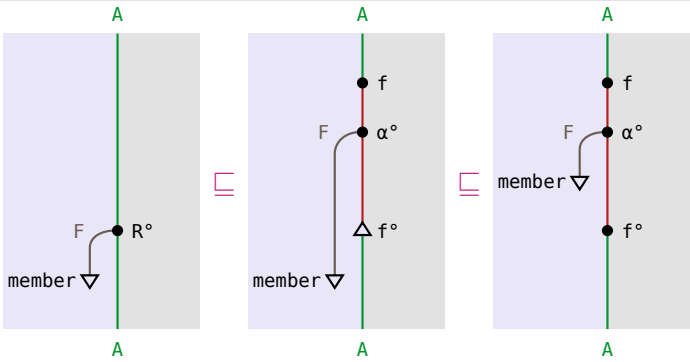


(a) (b)

(a)	f°	(6.5f) and ⦇R⦈f⊑𝟙
(b)	⊑ ⦇R⦈	

Map(f) ∧ R f ⊑ F(f) α ⟹ R° member ⊑ f α° member f° (6.5h)

R°member is below **α°member** conjugated by **f**, so it is inductive when **α°member** is



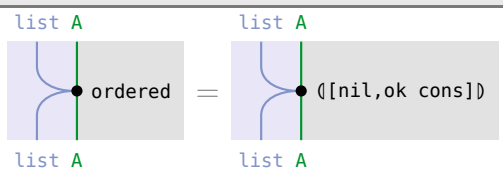
(a) (b) (c)

(a)	R° member	Map(f) ∧ R f ⊑ F(f) α ⟹ R° ⊑ f α° F(f°)
(b)	⊑ f α° F(f°) member	F(R) member ⊑ member R
(c)	⊑ f α° member f°	

§6.6. Sorting by selection

ordered = ⦇[nil,ok cons]⦈ (6.6a)

a list is sorted exactly when the fold that rebuilds it passes **ok** at every **cons**, i.e. each head is **R**-below every element after it

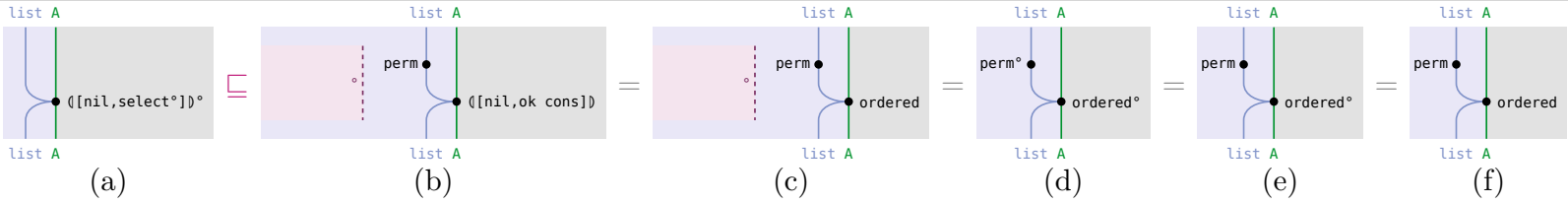


(a) (b)

(a)	ordered	fold uniqueness
(b)	= ⦇[nil,ok cons]⦈	

select° ⊑ ok cons perm ⟹ ⦇[nil,select°]⦈° ⊑ perm ordered (6.6b)

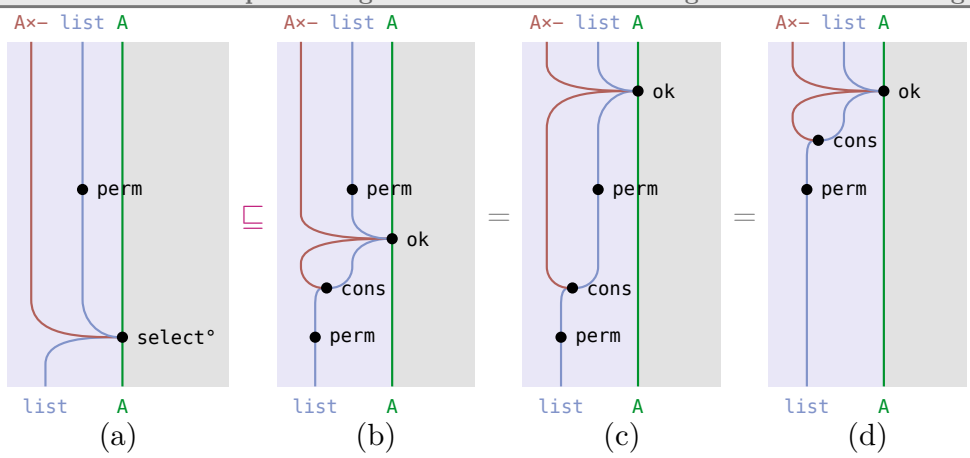
every output of unfolding the input by **select** is a sorted permutation of it



(a)	⦇[nil,select°]⦈°	select° ⊑ ok cons perm ⟹ ⦇[nil,select°]⦈° ⊑ (⦇[nil,ok cons]⦈ perm)°
(b)	⊑ (⦇[nil,ok cons]⦈ perm)°	ordered = ⦇[nil,ok cons]⦈
(c)	= (ordered perm)°	(R S)° = S° R°
(d)	= perm° ordered°	perm° = perm
(e)	= perm ordered°	ordered° = ordered
(f)	= perm ordered	

select° ⊑ ok cons perm ⟹ (𝟙 × perm) select° ⊑ ok cons perm (6.6c)

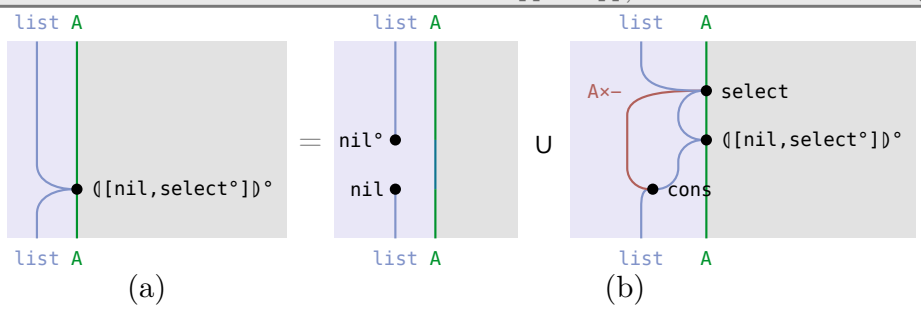
permuting the tail and then undoing **select** lands among the **ok** conses of a permutation



(a)	(𝟙 × perm) select°	select° ⊑ ok cons perm
(b)	⊑ (𝟙 × perm) ok cons perm	ok (𝟙 × perm) = (𝟙 × perm) ok
(c)	= ok (𝟙 × perm) cons perm	cons perm = (𝟙 × perm) cons perm
(d)	= ok cons perm	

⦇[nil,select°]⦈° = (nil° nil) ∪ (select (𝟙 × ⦇[nil,select°]⦈°) cons) (6.6d)

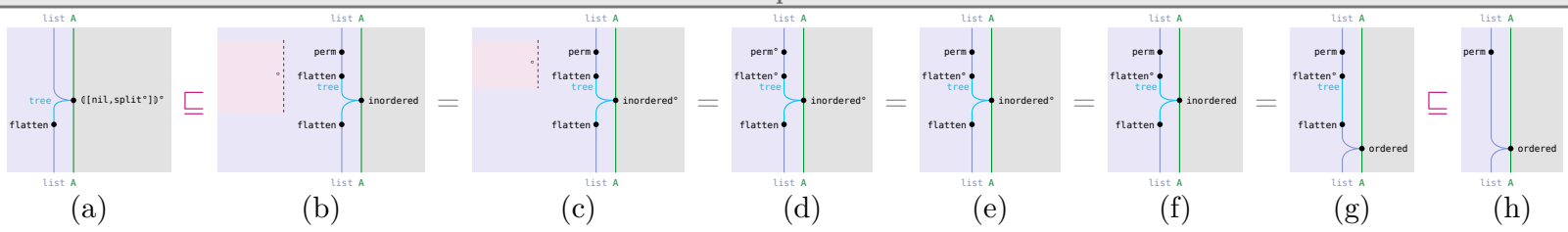
selection sort returns **[]** on **[]**, and otherwise selects **(a,y)**, sorts **y** and conses **a** back on



(a)	⦇[nil,select°]⦈°	unfolding the converse of a fold
(b)	= (nil° nil) ∪ (select (𝟙 × ⦇[nil,select°]⦈°) cons)	

(∀a. ∀b. ∀c. R(a,b) ⟹ R(b,c) ⟹ R(a,c)) ∧ split° ⊑ check' join perm ⟹ ⦇[nil,split°]⦈° flatten ⊑ perm ordered (6.6e)

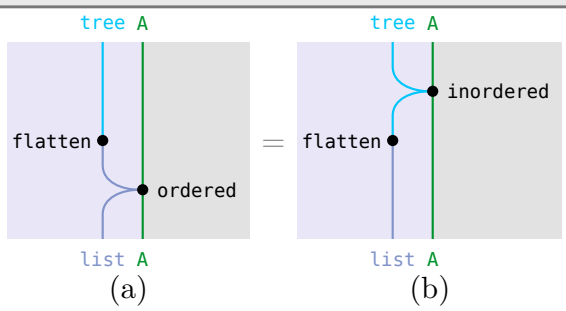
every output of unfolding the input by **split** into a tree and flattening that tree is a sorted permutation of the input



(a)	⦇[nil,split°]⦈° flatten	split° ⊑ check' join perm ⟹ ⦇[nil,split°]⦈° flatten ⊑ (inordered flatten perm)° flatten
(b)	⊑ (inordered flatten perm)° flatten	(R S)° = S° R°
(c)	= (flatten perm)° inordered° flatten	(R S)° = S° R°
(d)	= perm° flatten° inordered° flatten	perm° = perm
(e)	= perm flatten° inordered° flatten	inordered° = inordered
(f)	= perm flatten° inordered flatten	(∀a. ∀b. ∀c. R(a,b) ⟹ R(b,c) ⟹ R(a,c)) ⟹ flatten ordered = inordered flatten
(g)	= perm flatten° flatten ordered	flatten° flatten ⊑ 𝟙 𝟙 f = f
(h)	⊑ perm ordered	

(∀a. ∀b. ∀c. R(a,b) ⟹ R(b,c) ⟹ R(a,c)) ⟹ flatten ordered = inordered flatten (6.6f)

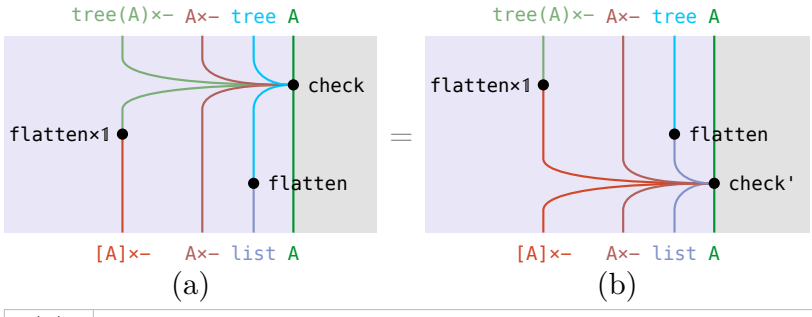
flattening a tree and then testing the list for order is the same as testing the tree with **inordered** and then flattening



(a)	flatten ordered	R transitive: a flattened tree is ordered iff every node passes check
(b)	= inordered flatten	

check (flatten × (𝟙 × flatten)) = (flatten × (𝟙 × flatten)) check' (6.6g)

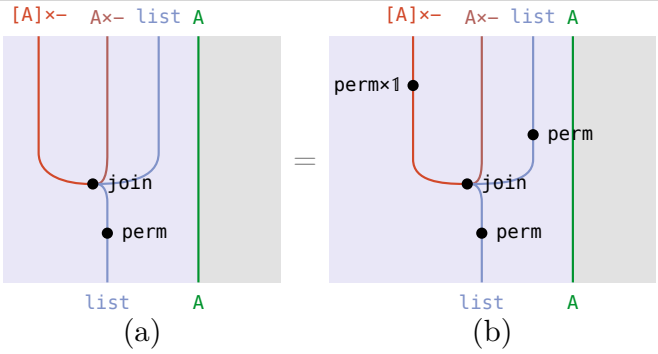
testing a fork with **check** and then flattening both subtrees equals flattening both subtrees and then testing with **check'**, since **intree** of a tree is **inlist** of its flattening



(a)	check (flatten × (𝟙 × flatten))	b intree x ⟺ b inlist flatten(x)
(b)	= (flatten × (𝟙 × flatten)) check'	

`join perm = (perm × (𝟙 × perm)) join perm` (6.6h)

permuting the list `x ++ [a] ++ y` is the same as first permuting `x` and `y` and then permuting the joined list

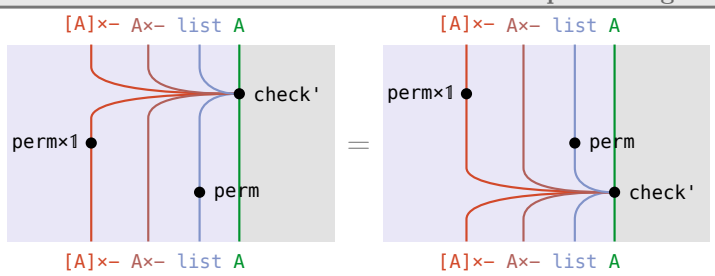


(a) (b)

(a)	`join perm`	`perm` is transitive and closed under `++`
(b)	`= (perm × (𝟙 × perm)) join perm`	

`check' (perm × (𝟙 × perm)) = (perm × (𝟙 × perm)) check'` (6.6i)

testing a fork of lists with `check'` and then permuting both parts equals permuting both parts and then testing, since permuting a list leaves its members unchanged

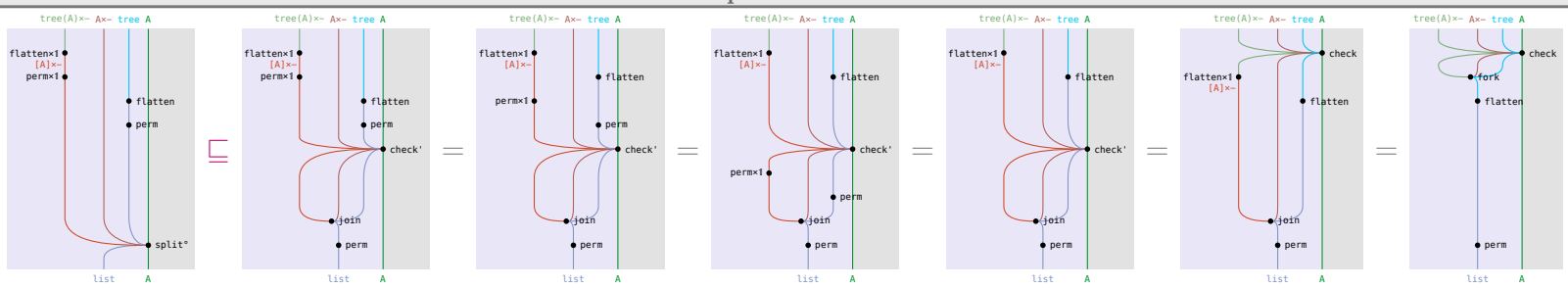


(a) (b)

(a)	`check' (perm × (𝟙 × perm))`	`b inlist x ⟺ b inlist perm(x)`
(b)	`= (perm × (𝟙 × perm)) check'`	

`split° ⊑ check' join perm ⟹ ((flatten perm) × (𝟙 × (flatten perm))) split° ⊑ check fork flatten perm` (6.6j)

undoing `split` and then flattening and permuting both parts lands among the `check`ed forks whose flattening is permuted

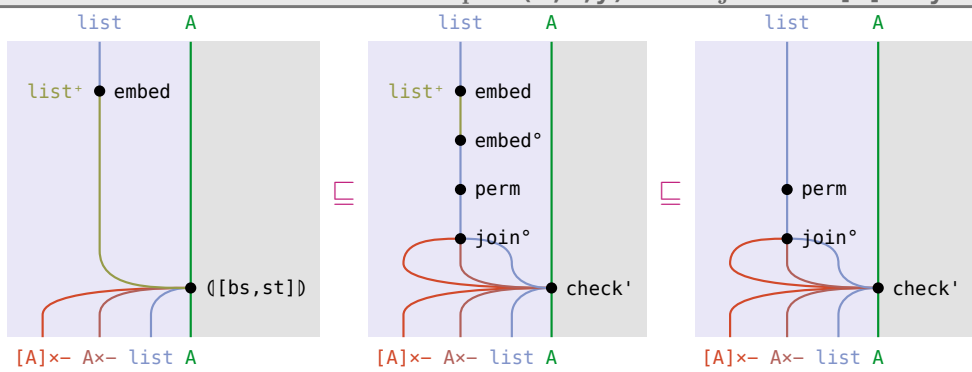


(a) (b) (c) (d) (e) (f) (g)

(a)	`((flatten perm) × (𝟙 × (flatten perm))) split°`	`split° ⊑ check' join perm`
(b)	`⊑ ((flatten perm) × (𝟙 × (flatten perm))) check' join perm`	`(f × (𝟙 × f)) (g × (𝟙 × g)) = (f g) × (𝟙 × (f g))`
(c)	`= (flatten × (𝟙 × flatten)) (perm × (𝟙 × perm)) check' join perm`	`check' (perm × (𝟙 × perm)) = (perm × (𝟙 × perm)) check'`
(d)	`= (flatten × (𝟙 × flatten)) check' (perm × (𝟙 × perm)) join perm`	`join perm = (perm × (𝟙 × perm)) join perm`
(e)	`= (flatten × (𝟙 × flatten)) check' join perm`	`check (flatten × (𝟙 × flatten)) = (flatten × (𝟙 × flatten)) check'`
(f)	`= check (flatten × (𝟙 × flatten)) join perm`	`fork flatten = (flatten × (𝟙 × flatten)) join`
(g)	`= check fork flatten perm`	

bs ⊑ wrap perm join° check' ∧ (𝟙 × (perm join° check')) st ⊑ cons perm join° check' ⟹ embed ⦇[bs,st]⦈ ⊑ perm join° check' (6.6k)

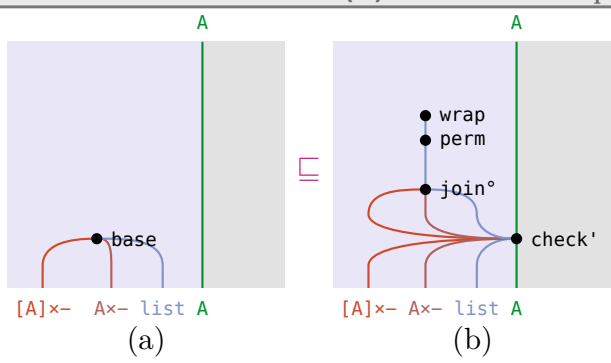
turning a list into a non-empty list and folding it with any `[bs, st]` meeting the two conditions below gives only `check`ed triples `(x,a,y)` whose join `x ⧺ [a] ⧺ y` is a permutation of the list



(a)	`embed ⦇[bs,st]⦈`	`bs ⊑ wrap perm join° check' ∧ (𝟙 × (perm join° check')) st ⊑ cons perm join° check' ⟹ embed ⦇[bs,st]⦈ ⊑ embed embed° perm join° check'`
(b)	`⊑ embed embed° perm join° check'`	`embed embed° perm join° check' ⊑ perm join° check'`
(c)	`⊑ perm join° check'`	

base ⊑ wrap perm join° check' (6.6l)

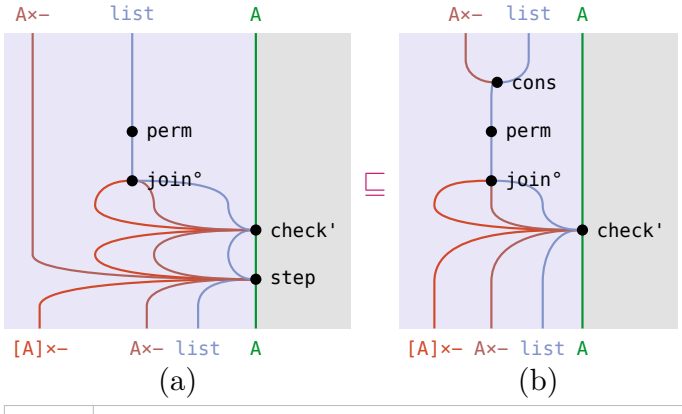
`base(a)` is a `check`ed triple whose join is a permutation of the one-element list `[a]`



(a)	`base`	`[] ⧺ [a] ⧺ [] = [a]`, and `check'` holds on empty lists
(b)	`⊑ wrap perm join° check'`	

(∀a. ∀b. leb(a,b) = true ⟹ R(a,b)) ∧ (∀a. ∀b. leb(a,b) = false ⟹ R(b,a)) ⟹ (𝟙 × (perm join° check')) step ⊑ cons perm join° check' (6.6m)

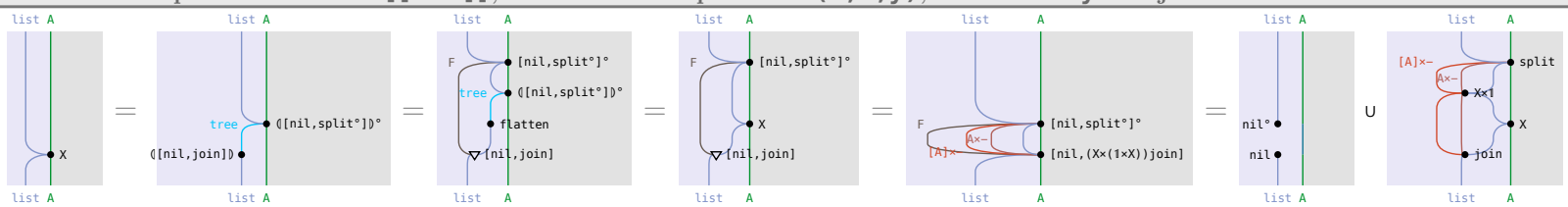
if `(x,b,y)` is a `check`ed triple for a permutation of `l`, then `step(a,(x,b,y))` is a `check`ed triple for a permutation of `[a] ⧺ l`



(a)	`(𝟙 × (perm join° check')) step`	`aRb` puts `a` in front of `x`; otherwise `bRa` and `a` goes in front of `y`
(b)	`⊑ cons perm join° check'`	

X = ⦇[nil,split°]⦈° flatten ⟹ X = (nil° nil) ∪ (split (X × (𝟙 × X)) join) (6.6n)

quicksort returns **[]** on **[]**, and otherwise splits into **(x,a,y)**, sorts **x** and **y** and joins them around **a**

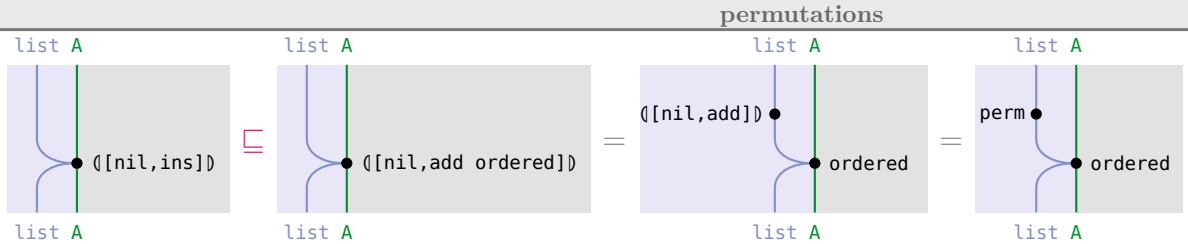


(a) (b) (c) (d) (e) (f)

(a)	X	X = ⦇[nil,split°]⦈° flatten
(b)	= ⦇[nil,split°]⦈° ⦇[nil,join]⦈	S° F(⦇S⦈° ⦇R⦈) R = ⦇S⦈° ⦇R⦈
(c)	= [nil,split°]° F(⦇[nil,split°]⦈° flatten) [nil,join]	X = ⦇[nil,split°]⦈° flatten
(d)	= [nil,split°]° F(X) [nil,join]	F(S) [T,U] = [T,(S × (𝟙 × S)) U]
(e)	= [nil,split°]° [nil,(X × (𝟙 × X)) join]	[U,V]° [R,S] = (U° R) ∪ (V° S)
(f)	= (nil° nil) ∪ (split (X × (𝟙 × X)) join)	

(𝟙 × ordered) ins ⊑ add ordered ⟹ ⦇[nil,ins]⦈ ⊑ perm ordered (6.6o)

for any **ins** that, on an ordered list, returns only ordered results of **add**, folding with **[nil, ins]** gives only sorted permutations

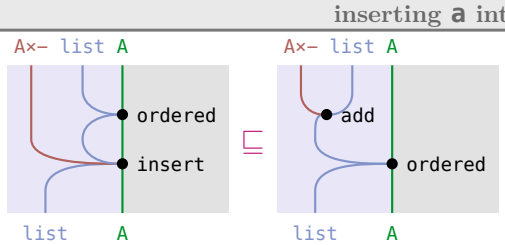


(a) (b) (c) (d)

(a)	⦇[nil,ins]⦈	(𝟙 × ordered) ins ⊑ add ordered ⟹ ⦇[nil,ins]⦈ ⊑ ⦇[nil,add ordered]⦈
(b)	⊑ ⦇[nil,add ordered]⦈	⦇[nil,add]⦈ ordered = ⦇[nil,add ordered]⦈
(c)	= ⦇[nil,add]⦈ ordered	perm = ⦇[nil,add]⦈
(d)	= perm ordered	

(∀a. ∀b. leb(a,b) = true ⟹ R(a,b)) ∧ (∀a. ∀b. leb(a,b) = false ⟹ R(b,a)) ∧ (∀a. ∀b. ∀c. R(a,b) ⟹ R(b,c) ⟹ R(a,c)) ⟹ (𝟙 × ordered) insert ⊑ add ordered (6.6p)

inserting **a** into an ordered list puts **a** somewhere in it and keeps it ordered



(a) (b)

(a)	(𝟙 × ordered) insert	insert splices a in, and R transitive and total
(b)	⊑ add ordered	

§6.7. Closure

R* = (μX : 𝟙 ∪ (X R)) (6.7a)

closing **R** by composing it on the left and closing it by composing on the right give the same relation **R***

𝟙 ⊑ X ∧ X X ⊑ X ⟹ R ⊑ X ⟺ R* ⊑ X (6.7b)

for a preorder **X**, **X** contains **R** exactly when it contains **R***

𝟙 ⊑ R* (6.7c)

R* is reflexive

(a)	𝟙	R ⊑ R ∪ S
(b)	⊑ 𝟙 ∪ (R R*)	𝟙 ∪ (R R*) = R*
(c)	= R*	

$R \sqsubseteq R^*$ (6.7d)

R^* contains R

(a)	R	$f\ \mathbb{1} = f$
(b)	$= R$	$\mathbb{1} \sqsubseteq R^*$
(c)	$\sqsubseteq R\ R^*$	$R\ R^* \sqsubseteq R^*$
(d)	$\sqsubseteq R^*$	

$R^*\ R^* \sqsubseteq R^*$ (6.7e)

R^* is transitive, because R^*/R^* is a prefixed point of $X \mapsto \mathbb{1} \cup RX$

(a)	$(\mathbb{1} \cup (R\ (R^* / R^*)))\ R^*$	$(P \cup Q)T = PT \cup QT$ — composition distributes over $\cup$
(b)	$= R^* \cup (R\ (R^* / R^*)\ R^*)$	$(S/S)S \sqsubseteq S$
(c)	$\sqsubseteq R^* \cup (R\ R^*)$	$RR^* \sqsubseteq R^*$ — (6.7d)
(d)	$\sqsubseteq R^*$	

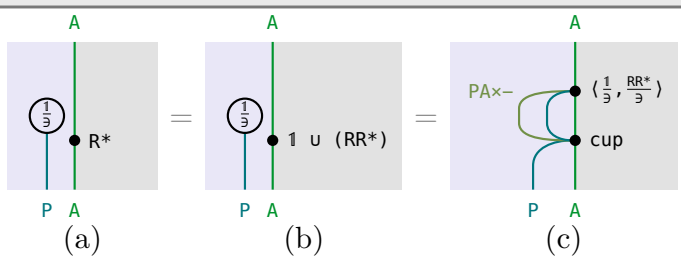
$\mathbb{1} \sqsubseteq X \wedge X\ X \sqsubseteq X \wedge R \sqsubseteq X \implies R^* \sqsubseteq X$ (6.7f)

a reflexive transitive X containing R is a prefixed point of $X \mapsto \mathbb{1} \cup RX$, so it contains R^*

(a)	$\mathbb{1} \cup (R\ X)$	$\mathbb{1} \sqsubseteq X$, $R \sqsubseteq X$
(b)	$\sqsubseteq X \cup (X\ X)$	$XX \sqsubseteq X$
(c)	$\sqsubseteq X$	

$\frac{R^*}{\ni} = \langle \frac{\mathbb{1}}{\ni}, \frac{R\ R^*}{\ni} \rangle\ \mathrm{cup}$ (6.7g)

the set of R^*-successors of a is a itself joined with the R^*-successors of its R-successors



(a)	$\frac{R^*}{\ni}$	$\mathbb{1} \cup (R\ R^*) = R^*$
(b)	$= \frac{\mathbb{1} \cup (R\ R^*)}{\ni}$	$\frac{R \cup S}{\ni} = \langle \frac{R}{\ni}, \frac{S}{\ni} \rangle\ \mathrm{cup}$
(c)	$= \langle \frac{\mathbb{1}}{\ni}, \frac{R\ R^*}{\ni} \rangle\ \mathrm{cup}$	

$R - \emptyset = R$ (6.7h)

taking nothing away leaves R

$R \cup (S - R) = R \cup S$

beside R, only the part of S outside R adds anything

$R - (S \cup T) = R - S - T$

taking away S and then T takes away $S \cup T$

$(R \cup S) - T = (R - T) \cup (S - T)$

taking T away from a union takes it away from each part

$\mathrm{monotonic}(\varphi) \wedge \mathrm{monotonic}(\psi) \implies (\mu X : \varphi(\psi(X))) = \varphi(\mu Y : \psi(\varphi(Y)))$ (6.7i)

the least fixed point of φ after ψ is φ applied to the least fixed point of ψ after φ

$S\ R^* = (\mu X : S \cup (X\ R))$ (6.7j)

S followed by any number of R steps is the least X containing S and closed under a further R step

$R^*\ S = (\mu X : S \cup (R\ X))$

any number of R steps followed by S is the least X containing S and closed under an R step in front

$\theta(\emptyset, S) = S\ R^*$ (6.7k)

started with nothing found and S to explore, θ computes SR^*

(a)	$\theta(\emptyset, S)$	
(b)	$\triangleq \emptyset \cup (\mu X : S \cup ((X\ R) - \emptyset))$	$R - \emptyset = R$
(c)	$= \emptyset \cup (\mu X : S \cup (X\ R))$	$\emptyset \cup R = R$
(d)	$= (\mu X : S \cup (X\ R))$	$S\ R^* = (\mu X : S \cup (X\ R))$
(e)	$= S\ R^*$	

θ(P,∅) = P (6.71)

with nothing left to explore, θ returns what it has found

(a)	θ(P,∅)	
(b)	≜ P ∪ (μX : ∅ ∪ ((X R) − P))	(μX : ∅ ∪ ((X R) − P)) = ∅
(c)	= P ∪ ∅	R ∪ ∅ = R
(d)	= P	